

HACKATHON • UNSTOP

FISAT

Project Ideas

12 Industry-Style Problems • Python Stack

Problem Statement

E-commerce platforms receive thousands of product images daily. Build a system that extracts attributes — weight, dimensions, expiry date, brand — from product images and outputs structured metadata automatically.

Real-World Use Case

A seller uploads a photo of a milk carton. The system reads the label, extracts '500ml', 'Expiry: Dec 2025', 'Brand: Amul', and auto-fills the product listing — saving hours of manual data entry.

Python Stack

- ▶ OpenCV / Pillow — image preprocessing
- ▶ Pytesseract — OCR text extraction
- ▶ spaCy / Regex — NLP attribute parsing
- ▶ FastAPI — REST API layer
- ▶ SQLite / JSON — structured output storage

Problem Statement

Develop a centralized platform to manage volunteers, food supplies, medicines, shelters, and emergency requests during disaster situations, with intelligent case prioritization.

Real-World Use Case

During a flood, relief coordinators log incoming emergency requests. The system auto-prioritizes critical medical cases, maps nearest available shelters, and notifies volunteer teams — replacing chaotic WhatsApp coordination.

Python Stack

- ▶ Flask / Django — backend web framework
- ▶ SQLAlchemy — DB models for resources
- ▶ Folium / Google Maps API — live mapping
- ▶ Celery + Redis — async alert dispatch
- ▶ Matplotlib — supply dashboard charts

3 Water Quality Learning & Certification Portal

EdTech

Problem Statement

Build a platform to monitor learner progress, manage certifications and assessments, and track learning outcomes for water quality management training programs.

Real-World Use Case

A government water board trains 500 field officers online. The portal tracks each officer's module completion, auto-generates certificates upon passing assessments, and provides department heads with a progress dashboard.

Python Stack

- ▶ Django — multi-role web portal
- ▶ Django REST Framework — API layer
- ▶ ReportLab — PDF certificate generation
- ▶ Chart.js via Python Jinja2 — dashboards
- ▶ SQLite / PostgreSQL — learner records

 Problem Statement

Create a system that predicts patient waiting times, manages OPD appointments, prioritizes emergency cases in real time, and provides live queue status to patients via web or SMS.

 Real-World Use Case

A hospital OPD has 200 daily walk-ins. The system predicts a 35-minute wait for token #47, instantly bumps a chest-pain patient to priority, and sends SMS updates — reducing patient anxiety and front-desk load.

 Python Stack

- ▶ Flask — lightweight API & queue engine
- ▶ APScheduler — appointment slot logic
- ▶ Twilio API — SMS queue notifications
- ▶ Pandas — wait-time prediction model
- ▶ Bootstrap + Jinja2 — patient-facing UI

Problem Statement

Automatically parse resumes, extract candidate skills and experience, match profiles against job descriptions using NLP, and rank applicants with explainable confidence scores.

Real-World Use Case

An HR team receives 800 applications for a data engineer role. The platform parses every resume in minutes, scores each candidate against the JD, and surfaces the top 20 — with highlighted skill matches — for human review.

Python Stack

- ▶ PyMuPDF / pdfplumber — resume parsing
- ▶ spaCy — skill & experience extraction
- ▶ Sentence-Transformers — JD matching
- ▶ scikit-learn — candidate ranking model
- ▶ Streamlit — recruiter dashboard UI

Problem Statement

Analyze traffic data from cameras or simulated sensor feeds to identify congestion hotspots, estimate vehicle density, and recommend dynamic signal timing adjustments.

Real-World Use Case

At a busy intersection, the system detects that northbound traffic has 3× the density of eastbound. It recommends extending the green phase by 20 seconds north, reducing average wait time by 40%.

Python Stack

- ▶ OpenCV — vehicle detection & counting
- ▶ YOLOv8 (ultralytics) — object detection
- ▶ NumPy / Pandas — traffic analytics
- ▶ Folium — congestion heatmap overlay
- ▶ Flask — real-time dashboard & API

7 NGO Resource & Donation Management

Social Impact

Problem Statement

Manage donations, inventory, volunteers, beneficiaries, and fund utilization through a centralized platform with reporting and transparency features for donors.

Real-World Use Case

An NGO running 10 community kitchens tracks rice stocks across locations, logs volunteer shifts, and auto-generates monthly fund utilization reports — giving donors real-time visibility into how their money is used.

Python Stack

- ▶ Django — multi-branch NGO portal
- ▶ Django Admin — inventory & donor CRUD
- ▶ ReportLab — fund utilization PDF reports
- ▶ Pandas — donation analytics & trends
- ▶ Chart.js + Jinja2 — impact dashboards

8 Smart College Placement Management Portal

Education

Problem Statement

Develop a centralized portal for managing students, companies, drives, eligibility filtering, interview scheduling, and placement analytics for a college placement cell.

Real-World Use Case

The placement officer lists an Infosys drive with a 7.5 CGPA cutoff. The system auto-filters eligible students, sends them notifications, records interview results, and updates the placement statistics dashboard live.

Python Stack

- ▶ Django — student / company / admin roles
- ▶ Celery — email & notification dispatch
- ▶ Pandas — analytics & placement reports
- ▶ OpenPyXL — Excel export for admin use
- ▶ Bootstrap + Django Templates — clean UI

Problem Statement

Build an ML system that classifies news articles as genuine or potentially misleading, provides a confidence score with reasoning, and visualizes the classification evidence for users.

Real-World Use Case

A user pastes a viral WhatsApp article claiming a new vaccine ban. The system flags it as '87% likely misleading', highlights the inflammatory language patterns, and cross-references known fact-check databases.

Python Stack

- ▶ scikit-learn / XGBoost — classification
- ▶ NLTK / spaCy — text preprocessing
- ▶ Hugging Face Transformers — BERT model
- ▶ LIME — explainability & reasoning output
- ▶ Streamlit — article submission & results UI

Problem Statement

Develop a secure role-based platform for incident reporting, multi-agency case tracking, authority coordination, and rehabilitation monitoring for child welfare organizations.

Real-World Use Case

A social worker files a neglect report. The system assigns a case ID, notifies the district child protection officer, tracks home visit schedules, court dates, and rehabilitation milestones — all in one auditable trail.

Python Stack

- ▶ Django — secure multi-role portal
- ▶ Django Guardian — object-level permissions
- ▶ Celery — deadline reminders & escalation
- ▶ ReportLab — confidential case PDF reports
- ▶ PostgreSQL — encrypted case record storage

 Problem Statement

Build a dashboard that ingests security logs, detects anomalies and suspicious activities using ML, visualizes attack trends in real time, and alerts administrators with prioritized alerts.

 Real-World Use Case

A university network generates 50,000 log lines per hour. The system flags 12 suspicious login attempts from an unusual IP, clusters them as a brute-force pattern, and emails the security team — before any breach occurs.

 Python Stack

- ▶ Pandas — log ingestion & preprocessing
- ▶ scikit-learn — anomaly detection (IsolationForest)
- ▶ Plotly Dash — real-time threat dashboard
- ▶ smtplib — automated admin alert emails
- ▶ SQLite — alert history & event logging

 Problem Statement

Build an AI-powered chatbot that helps citizens find information about government schemes, check eligibility, understand application procedures, and get directed to the right departments.

 Real-World Use Case

A farmer asks: 'Am I eligible for PM-KISAN?' The chatbot collects land ownership details, checks eligibility criteria, explains the application steps in Tamil or English, and provides the nearest CSC center location.

 Python Stack

- ▶ LangChain + Ollama — local LLM pipeline
- ▶ FAISS — vector DB for scheme documents
- ▶ spaCy — intent detection & slot filling
- ▶ Flask — chatbot REST API backend
- ▶ Streamlit / Gradio — citizen-facing chat UI

HACKATHON FISAT • UNSTOP

Build Something That Matters.

12 problems. 1 Python stack. Infinite possibilities.

Choose your problem. Build the prototype. Ship it.